Program:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split

# Example text data and labels
texts = [
    "I love this movie",
    "This film is amazing",
    "The plot was predictable",
    "The acting was terrible"
]
labels = np.array([1, 1, 0, 0])  # 1 for positive sentiment, 0 for negative sentiment

# Tokenize the text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
vocab_size = len(tokenizer.word_index) + 1
sequences = tokenizer.texts_to_sequences(texts)

# Pad sequences to have the same length
max_sequence_length = max([len(seq) for seq in sequences])
padded_sequences = pad_sequences(sequences, maxlen=max_sequence_length)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(padded_sequences, labels, test_size=0.2)

# Define the CNN model
model = Sequential()
model.add(Embedding(vocab_size, 10, input_length=max_sequence_length))
model.add(Conv1D(64, 5, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=10, batch_size=1)
```

```
# Evaluate the model on the testing set
loss, accuracy = model.evaluate(X_test, y_test)
print("Test loss:", loss)
print("Test accuracy:", accuracy)

# Example text for prediction
new_texts = [
    "This movie is fantastic",
    "I hated the acting in this film"
]

# Tokenize and pad new texts
new_sequences = tokenizer.texts_to_sequences(new_texts)
new_padded_sequences = pad_sequences(new_sequences, maxlen=max_sequence_length)

# Perform prediction
predictions = model.predict(new_padded_sequences)

# Print predictions
for i in range(len(new_texts)):
    print(f"Text: {new_texts[i]}")
    if predictions[i] > 0.5:
        print("Sentiment: Positive")
    else:
        print("Sentiment: Negative")
    print()
```

Output:

```
Epoch 1/10
3/3 [==============================] - 1s 18ms/step - loss: 0.6934 - accuracy:
0.3333
Epoch 2/10
3/3 [==============================] - 0s 17ms/step - loss: 0.6856 - accuracy:
0.6667
Epoch 3/10
3/3 [==============================] - 0s 17ms/step - loss: 0.6786 - accuracy:
0.6667
Epoch 4/10
3/3 [==============================] - 0s 17ms/step - loss: 0.6717 - accuracy:
1.0000
Epoch 5/10
3/3 [==============================] - 0s 18ms/step - loss: 0.6643 - accuracy:
1.0000
Epoch 6/10
3/3 [==============================] - 0s 17ms/step - loss: 0.6561 - accuracy:
1.0000
Epoch 7/10
3/3 [==============================] - 0s 18ms/step - loss: 0.6469 - accuracy:
1.0000
Epoch 8/10
3/3 [==============================] - 0s 18ms/step - loss: 0.6362 - accuracy:
1.0000
Epoch 9/10
3/3 [==============================] - 0s 17ms/step - loss: 0.6237 - accuracy:
1.0000
Epoch 10/10
3/3 [==============================] - 0s 18ms/step - loss: 0.6090 - accuracy:
1.0000
1/1 [==============================] - 0s 64ms/step - loss: 0.5737 - accuracy:
1.0000
Test loss: 0.5737178320884705
Test accuracy: 1.0
Text: This movie is fantastic
Sentiment: Positive

Text: I hated the acting in this film
Sentiment: Negative
```